

search.py

```
import time
```

برای محاسبه زمان اجرا الگوریتم ها و استفاده برای مقایسه

جزء	نقش	نوع داده
fringe	نگهداری گره‌های منتظر بررسی	(صف) و پشته و صف الویت دار
explored	گره‌هایی که قبلاً بررسی شده‌اند	Set و دیکشنری
path	مسیر از start تا گره فعلی	list
counter	شمارش ترتیب گره ها	متغیر int

در الگوریتم هایی که هزینه را ملاک عمل خود قرار می دهد باید چک کند که در صورتی که گره ای پیدا شود که قبلاً پیدا شده ولی گسترش نیافته ولی اینبار با هزینه کمتر نسبت به قبل , پس باید جایگزین قبلی شود ولی در اینجا به این موضوع نپرداختیم در واقع ما فقط چک کردیم که گره گسترش یافته دوباره وارد صف نشود و در صورت نبودن هدف به صف اضافه شود این یعنی ممکنه گره ای چندین بار با هزینه های مختلف وارد صف شود و ما چیزی را حذف و یا جایگزین نمی کنیم دلیلش هم این است که ما با صف الویت دار کمترین را برای گسترش انتخاب می کنیم و بعد از گسترش دیگر آن گره نمی تواند گسترش یابد و گره ها با هزینه بیشتر خودبخود حذف می شوند و گسترش نمی یابند

توضیحات راجب توابع search.py

BFS

محیط را لایه به لایه جستجو می‌کند. یعنی اول همسایه‌های شروع، سپس همسایه‌های آنها، و الی آخر.

```
if start == goal:  
    return [start]
```

اگر نقطه شروع همان نقطه هدف باشد، مسیر فقط شامل همان یک نقطه است

چون نقاط شروع و پایان رندم انتخاب می شوند احتمال دارد هر دو در یک نقطه باشند .

```
start_time = time.time()
```

آغاز شمارش زمان

deque

یک ساختمان داده است که از دو طرف می‌توان به آن اضافه یا حذف کرد

برای تشکیل صف ازش استفاده شده برای اجرا کردن FIFO

fringe

دریک تابل از خود گره و مسیر گره را نگه می‌دارد

while fringe:

حلقه تازمان پر بودن صف

Node, path = fringe.popleft()

اولی که وارد شده را بردار

if Node in explored:

continue

برای جلوگیری از تشکیل حلقه و گسترش چند باره گره‌ها

explored.add(Node)

گسترش گره‌ای که قبلاً گسترش نیافته

for neighbor in neighbors_fn(Node):

if neighbor not in explored:

if neighbor == goal:

end_time = time.time()

print(f"BFS - Nodes expanded: {len(explored)}")

print(f"BFS - Path length: {len(path) + 1}")

print(f"BFS - Time: {(end_time - start_time)*1000:.2f} ms")

return path + [neighbor]

fringe.append((neighbor, path + [neighbor]))

گرفتن همسایه‌های گره، همسایه‌ها نباید قبلاً گسترش یافته باشند

تست هدف در هنگام تولید و در صورت یافتن هدف پایان شمارش زمان و چاپ اطلاعات برگرداندن مس
یر گره + مقعیت هدف یافت شده در صورت نبودن هدف اضافه به لیست پیدا شده‌ها تا بعداً گسترش یابند

end_time = time.time()

print(f"BFS - No path found! Nodes expanded: {len(explored)}")

print(f"BFS - Time: {(end_time - start_time)*1000:.2f} ms")

return []

برای زمانی که نقطه پایانی یافت نشد

DFS

```
if start == goal:  
    return [start]
```

اگر نقطه شروع همان نقطه هدف باشد، مسیر فقط شامل همان یک نقطه است
چون نقاط شروع و پایان رندم انتخاب می شوند احتمال دارد هر دو در یک نقطه باشند .

```
start_time = time.time()
```

آغاز شمارش زمان

```
fringe = [(start, [start])]  
explored_path = {}
```

استفاده از پشته برای مفهوم LIFO برای پیدا شده ها

استفاده از دیکشنری برای مسیر های گسترش یافته

در اینجا ما گره های گسترش یافته را نمی خواهیم بلکه مسیر گسترش را می خواهیم چون به عمق می
زنیم و مسیر ها را چک می کنیم

```
while fringe:
```

حلقه تازمان پر بودن پشته

```
Node, path = fringe.pop()
```

آخری که وارد شده را می دهد

```
explored_path[Node] = path
```

گسترش گره همراه با مسیراش

```
for neighbor in neighbors_fn(Node):
```

```
    if neighbor not in path:
```

```
        if neighbor == goal:
```

```
            end_time = time.time()
```

```
            print(f"DFS - Nodes expanded: {len(explored_path)}")
```

```
            print(f"DFS - Path length: {len(path) + 1}")
```

```
            print(f"DFS - Time: {(end_time - start_time)*1000:.2f} ms")
```

```
            return path + [neighbor]
```

```
fringe.append((neighbor, path + [neighbor]))
```

گرفتن همسایه های گره , مسیر همسایه ها نباید قبلا گسترش یافته باشند
تست هدف در هنگام تولید و در صورت یافتن هدف پایان شمارش زمان و چاپ اطلاعات برگرداندن مس
یر گره + مقعیت هدف یافت شده در صورت نبودن هدف اضافه به لیست پیدا شده ها تا بعدا گسترش یابند

```
end_time = time.time()
print(f'DFS - No path found! Nodes expanded: {len(explored_path)}')
print(f'DFS - Time: {(end_time - start_time)*1000:.2f} ms")
return []
```

برای زمانی که نقطه پایانی یافت نشد

UCS

```
if start == goal:
    return [start]
```

اگر نقطه شروع همان نقطه هدف باشد، مسیر فقط شامل همان یک نقطه است
چون نقاط شروع و پایان رندم انتخاب می شوند احتمال دارد هر دو در یک نقطه باشند .

```
start_time = time.time()
```

آغاز شمارش زمان

```
counter = 0
fringe = [(0, counter, start, [start])]
explored = {}
```

ایجاد متغیر برای شمارش الویت گره ها ذخیره تاپل نقطه شروع در پیدا شده ها ایجاد دیکشنری گسترش
یافته ها

```
while fringe:
```

حلقه تازمان پر بودن صف الویت

```
cost, _, Node, path = heapq.heappop(fringe)
```

یک صف الویت دار اینگونه که اولین عضو را در نظر می گیرد و کمترین مقدار را بر می گردانند یعنی
هزینه کمتر در صورت وجود دو یا چند گره با هزینه یکسان به سراغ عنصر دوم یعنی counter می
رود و آنکه الویت کمتر دارد را بر می گردانند آن گره که زود تر پیدا شده و وارد لیست پیدا شده ها
شده یعنی استراژی ما اینجا برای هزینه برابر FIFO هست

```
if Node in explored:
    continue
```

برای جلو گیری از تشکیل حلقه و گسترش چند باره گره ها

```
explored[Node] = cost
```

ذخیره گره گسترش یافته با هزینه خود (هزینه خود + هزینه مسیر قبل از خود)

```
if Node == goal:
```

```
    end_time = time.time()
```

```
    print(f"UCS - Nodes expanded: {len(explored)}")
```

```
    print(f"UCS - Total cost: {cost}")
```

```
    print(f"UCS - Path length: {len(path)}")
```

```
    print(f"UCS - Time: {(end_time - start_time)*1000:.2f} ms")
```

```
    return path
```

تست هدف در زمان گسترش ، پیدا کردن هدف ، پایان شمارش ، زمان چاپ اطلاعات و برگرداندن مسیر

```
for neighbor, step_cost in neighbors_fn(Node):
```

```
    if neighbor not in explored:
```

```
        new_cost = cost + step_cost
```

```
        counter += 1
```

```
        heapq.heappush(fringe, (new_cost, counter, neighbor, path + [neighbor]))
```

پیدا کردن همسایه های گره نباید قبلاً گسترش یافته باشد بدست آوردن هزینه واقعی گره (هزینه گره و مسیر قبل از آن) افزایش شماره الویت و ذخیره گره در پیدا شده ها

```
end_time = time.time()
```

```
print(f"UCS - No path found! Nodes expanded: {len(explored)}")
```

```
print(f"UCS - Time: {(end_time - start_time)*1000:.2f} ms")
```

```
return []
```

برای زمانی که نقطه پایانی یافت نشد

A*

```
if start == goal:
```

```
    return [start]
```

اگر نقطه شروع همان نقطه هدف باشد، مسیر فقط شامل همان یک نقطه است

چون نقاط شروع و پایان رندم انتخاب می شوند احتمال دارد هر دو در یک نقطه باشند .

```
start_time = time.time()
```

آغاز شمارش زمان

```
if heuristic == "manhattan":
```

```
    h_cost = manhattan
```

else:

h_cost = euclidean

بدست آوردن هیروستیک طبق خواسته کاربر

g_cost = {start: 0}

counter = 0

fringe = [(h_cost(start, goal), counter, start, [start])]

explored = {}

هزینه طی شده برای نقطه شروع صفر شماره الویت صفر ذخیره نقطه شروع در پیداشده ها و ایجاد دیکشنری گسترش یافته ها

while fringe:

حلقه تازمان پر بودن صف الویت

cost, _, Node, path = heapq.heappop(fringe)

انتخاب کمترین هزینه از پیدا شده ها در صورت پیدا شدن دو و یا چند گره با هزینه برابر انتخاب کمترین عنصر دوم (شماره الویت counter) از آنجا که الگوریتم A* در صورت پیدا شدن گره هم هزینه در مسیر کاوش خود مسیر را رها نمی کند و مسیر را ادامه می دهد انگار که دارد در انتخاب گره های هم هزینه گره جدید را به گره قدیم ترجیح می دهد LIFO پس برای آنکه گره جدید تر نسبت به گره قدیمی تر الویت داشته باشد برای شمارش الویت counter گره ها بجای اعداد مثبت از اعداد منفی استفاده می شود چون با افزایش مقدار منفی عدد کوچکتر می شود و در انتخاب صف الویت زودتر انتخاب می شود

if Node in explored:

continue

برای جلوگیری از تشکیل حلقه و گسترش چند باره گره ها

explored[Node] = cost

ذخیره گره گسترش یافته با هزینه خود (هزینه طی شده + هزینه تخمین زده مسیر مانده)

if Node == goal:

end_time = time.time()

print(f"A* ({heuristic}) - Nodes expanded: {len(explored)}")

print(f"A* ({heuristic}) - Total cost: {g_cost[Node]}")

print(f"A* ({heuristic}) - Path length: {len(path)}")

print(f"A* ({heuristic}) - Time: {(end_time - start_time)*1000:.2f} ms")

return path

تست هدف در زمان گسترش , پیدا کردن هدف , پایان شمارش زمان , چاپ اطلاعات و برگرداندن مسیر

```
for neighbor, step_cost in neighbors_fn(Node):
    if neighbor in explored:
        continue
    else:
        g_cost[neighbor] = g_cost[Node] + step_cost
        f_cost = g_cost[neighbor] + h_cost(neighbor, goal)
        counter -= 1
        heapq.heappush(fringe, (f_cost, counter, neighbor, path + [neighbor]))
```

پیدا کردن همسایه ها , همسایه نباید قبلا گسترش یافته باشد بدست آوردن هزینه طی شده هزینه مسیر های قبل + هزینه گره جدید , بدست آوردن هزینه واقعی یعنی مجموع هزینه طی شده و هزینه تخمین خورده برای باقی مسیر تا هدف , کم کردن الویت شمارش , گره جدیدتر الویت بیشتری دارند , ذخیره گره

```
end_time = time.time()
print(f'A* ({heuristic}) - No path found! Nodes expanded: {len(explored)}"
)
print(f'A* ({heuristic}) - Time: {(end_time - start_time)*1000:.2f} ms")
return []
```

برای زمانی که نقطه پایانی یافت نشد

```
def manhattan(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
def euclidean(a, b):
    return math.sqrt((a[0] - b[0])**2 + (a[1] - b[1])**2)
```

شیوه محاسبه مسیر باقی مانده تا هدف

$$\text{manhattan} = |X1 - X2| + |Y1 - Y2|$$

جمع فاصله عمودی و افقی مبدا تا مقصد

$$\text{Euclidean} = \sqrt{(X1 - X2)^2 + (Y1 - Y2)^2}$$

فاصله مستقیم از مبدا تا مقصد

grid_env.py

برای مقایسه الگوریتم‌ها بهتر بود که بجای اینکه هر بار محیط ریستارت بشود از یک محیط برای همه الگوریتم‌ها استفاده شود و داده‌ها با هم مقایسه بشوند برای همین تغییراتی در این کد دادم

```
elif choice == "6": print("\n Running ALL algorithms on same map \n") algos = [("DFS", False, None), ("BFS", False, None), ("UCS", True, None), ("A* manhattan", True, "manhattan"), ("A* euclidean", True, "euclidean")] for name, w, h in algos: print(f"\n~~~~~ {name} ~~~~~") if w: if h: p = a_star_search(start, goal, get_neighbors_fn(grid, moving_obstacles, True), heuristic=h) else: p = uniform_cost_search(start, goal, get_neighbors_fn(grid, moving_obstacles, True)) else: p = depth_first_search(start, goal, get_neighbors_fn(grid, moving_obstacles, False)) if name=="DFS" else breadth_first_search(start, goal, get_neighbors_fn(grid, moving_obstacles, False)) input("\nPress Enter for next algorithm...") continue
```

از این تابع برای مقایسه الگوریتم‌ها استفاده کردم

تحلیل داده

سری اول اجر

الگوریتم	تعداد گره‌های گسترش یافته	طول مسیر	هزینه کل	زمان (ms)
DFS	3	4	-	0.04
BFS	9	4	-	0.08
UCS	9	4	3	9.42
* (منهتن) A	4	4	3	0.15
* (قلیسی) A	4	4	3	0.06

تحلیل نتایج:

🏆 DFS: بهترین الگوریتم از نظر زمان

- (سریع‌ترین) **0.04 ms**: زمان
- فقط 3 گره گسترش داده

🏆 UCS, A*: بهترین الگوریتم از نظر هزینه

- (بهینه) 3: هزینه

🏆 DFS, A*: بهترین الگوریتم از نظر گره گسترش یافته

- (بسیار کم) گره 3-4 فقط

سری دوم

الگوریتم	تعداد گره‌های گسترش یافته	طول مسیر	هزینه کل	زمان (ms)
DFS	165	108	-	24.19
BFS	4	4	-	0.03
UCS	8	4	3	0.06
<i>A* (منهتن)</i>	4	4	3	0.04
<i>A* (قلیدسی)</i>	4	4	3	0.05

تحلیل نتایج:

🏆 BFS: بهترین الگوریتم از نظر زمان

(سریع‌ترین) ms زمان: 0.03

فقط 4 گره گسترش داده

🏆 *A* (قلیدسی)*, *A* (منهتن)*, UCS: بهترین الگوریتم از نظر هزینه

هزینه: 3 (بهینه)

هر سه الگوریتم هزینه بهینه را پیدا کردند

**** (اقلیدسی) A, (منهتن) A, BFS: بهترین الگوریتم از نظر گره گسترش یافته 🏆**
فقط 4 گره (بسیار کم)

DFS: ضعیف‌ترین الگوریتم ✖

(کندترین) ms زمان: 24.19

گره گسترش یافته: 165 (بیشترین)

طول مسیر: 108 قدم (بلندترین)

تحلیل دو سری برای الگوریتم‌ها

DFS

در محیط ساده: ✔ عملکرد خوب

در محیط پیچیده: ✖ عملکرد فاجعه‌بار

نتیجه. ناپایدارترین الگوریتم است. برای محیط‌های پیچیده توصیه نمی‌شود DFS

BFS

در هر دو محیط: ✔ کوتاه‌ترین مسیر (4 قدم)

در محیط پیچیده: ✔: **بهتر و سریع‌تر** عمل کرد

نتیجه BFS: **پایدار و قابل اعتماد** است. برای یافتن کوتاه‌ترین مسیر عالی است.

UCS

(3) در هر دو محیط: ✔ هزینه بهینه

(9.42ms) در سری اول: ✖ زمان بسیار بالا

(0.06ms) در سری دوم: ✔ زمان عالی

نتیجه. عملکرد ناپایدار از نظر زمان دارد ولی هزینه بهینه را تضمین می‌کند UCS

A* Manhattan

در هر دو محیط ✓: تعداد گره ثابت 4 گره

در هر دو محیط: ✓ هزینه بهینه (3)

زمان در سری دوم ✓: بهبود یافت

نتیجه A*: منتهن پایدار و عالی است.

A* Euclidean

تعداد گره ثابت ✓: گره 4 در هر دو محیط

هزینه بهینه (3) در هر دو محیط: ✓

زمان = (0.05-0.06ms) پایدارترین الگوریتم ✓

نتیجه. اقلیدسی پایدارترین و قابل اعتمادترین الگوریتم است A*

رتبه	الگوریتم	امتیاز	دلیل
1	<i>A (Euclidean)*</i>	100	پایدارترین، همیشه بهینه، زمان عالی
2	<i>A (Manhattan)*</i>	98	پایدار، همیشه بهینه، زمان عالی
3	BFS	90	پایدار، سریع، ولی هزینه را نمی‌فهمد
4	UCS	70	هزینه بهینه ولی زمان ناپایدار

5	DFS	30	ناپایدار، در محیط پیچیده شکست می‌خورد
---	------------	----	---------------------------------------

تصاویر داده های خروجی در PDF تصاویر موجود است

دلیل استفاده از هرگونه هوش مصنوعی

برای تحلیل داده پیدا کردن ساختمان داده مناسب برای هر الگوریتم مثل صف پشته و یا صف الویت دار و همچنین برای درک بهتر `grid_env.py`